

OVER Clause in SQL Server with Examples

In this article, I am going to discuss the **OVER Clause in SQL Server** with examples. Please read our previous article where we discussed the [built-in string function](#) in SQL Server. At the end of this article, you will understand the power and use of the OVER Clause in SQL Server with Examples.

OVER Clause in SQL Server:

The OVER clause in SQL Server is used with PARTITION BY to break up the data into partitions. Following is the syntax of the OVER clause.

```
<function> OVER (  
    [PARTITION BY clause]  
    [ORDER BY clause]  
    [ROWS or RANGE clause])
```

The specified function is going to operate for each partition. See the following example. Let say we have three departments (HR, IT, Payroll).

COUNT (Department) OVER (PARTITION BY Department)

In the above example, the data will be partition by Department i.e. there will be three partitions (IT, HR, and Payroll), and then the COUNT() function will be applied to each partition. Here, you can use a wide range of built-in functions such as COUNT(), SUM(), MAX(), ROW_NUMBER(), RANK(), DENSE_RANK(), AVG(), MIN(), etc.

Example: OVER clause in SQL Server

Let us see an example, to understand the use of the SQL Server Over clause. We are going to use the following Employee table.

ID	Name	Department	Salary
1	James	IT	15000
2	Smith	IT	35000
3	Rasol	HR	15000
4	Rakesh	Payroll	35000
5	Pam	IT	42000
6	Stokes	HR	15000
7	Taylor	HR	67000
8	Preety	Payroll	67000
9	Priyanka	Payroll	55000
10	Anurag	Payroll	15000
11	Marshal	HR	55000
12	David	IT	96000

Please use the following SQL Script to create and populate the Employees table with the required data.

```
CREATE TABLE Employees
(
    ID INT,
    Name VARCHAR(50),
    Department VARCHAR(50),
    Salary int
)
Go
INSERT INTO Employees Values (1, 'James', 'IT', 15000)
INSERT INTO Employees Values (2, 'Smith', 'IT', 35000)
INSERT INTO Employees Values (3, 'Rasol', 'HR', 15000)
INSERT INTO Employees Values (4, 'Rakesh', 'Payroll', 35000)
INSERT INTO Employees Values (5, 'Pam', 'IT', 42000)
INSERT INTO Employees Values (6, 'Stokes', 'HR', 15000)
INSERT INTO Employees Values (7, 'Taylor', 'HR', 67000)
INSERT INTO Employees Values (8, 'Preety', 'Payroll', 67000)
INSERT INTO Employees Values (9, 'Priyanka', 'Payroll', 55000)
INSERT INTO Employees Values (10, 'Anurag', 'Payroll', 15000)
INSERT INTO Employees Values (11, 'Marshal', 'HR', 55000)
INSERT INTO Employees Values (12, 'David', 'IT', 96000)
```

Example:

We need to generate a report to display the total number of employees department-wise. Along with this we also need to display the Total Salary, Average Salary, Minimum Salary, and Maximum Salary department wise. That means we need to generate a report like below.

Department	NoOfEmployees	TotalSalary	AvgSalary	MinSalary	MaxSalary
HR	4	152000	38000	15000	67000
IT	4	188000	47000	15000	96000
Payroll	4	172000	43000	15000	67000

We can easily achieve the above data simply by using the GROUP BY clause in SQL Server. The following SQL Query will give you the desired output.

```
SELECT Department,
COUNT(*) AS NoOfEmployees,
SUM(Salary) AS TotalSalary,
AVG(Salary) AS AvgSalary,
MIN(Salary) AS MinSalary,
MAX(Salary) AS MaxSalary
FROM Employees
GROUP BY Department
```

Example:

Now the business requirement changes, now we also need to show the non-aggregated values (Name and Salary) in the report along with the aggregated values as shown in the below image.

Name	Salary	Department	DepartmentTotals	TotalSalary	AvgSalary	MinSalary	MaxSalary
Rasol	15000	HR	4	152000	38000	15000	67000
Stokes	15000	HR	4	152000	38000	15000	67000
Taylor	67000	HR	4	152000	38000	15000	67000
Marshal	55000	HR	4	152000	38000	15000	67000
David	96000	IT	4	188000	47000	15000	96000
Pam	42000	IT	4	188000	47000	15000	96000
James	15000	IT	4	188000	47000	15000	96000
Smith	35000	IT	4	188000	47000	15000	96000
Rakesh	35000	Payroll	4	172000	43000	15000	67000
Preety	67000	Payroll	4	172000	43000	15000	67000
Priyanka	55000	Payroll	4	172000	43000	15000	67000
Anurag	15000	Payroll	4	172000	43000	15000	67000

You may be intended to use the following SQL query by adding the Salary, Name column in the select clause. But this is not going to be work.

```
SELECT Name, Salary, Department,
COUNT(*) AS NoOfEmployees,
SUM(Salary) AS TotalSalary,
AVG(Salary) AS AvgSalary,
```

```

MIN(Salary) AS MinSalary,
MAX(Salary) AS MaxSalary
FROM Employees
GROUP BY Department

```

When you execute the above query, you will get the following error. This is because it is not possible to include the non-aggregated column in the select list when you are using a group by clause in SQL Server.

Msg 8120, Level 16, State 1, Line 23
Column 'Employees.Name' is invalid in the select list because it is not contained in either an aggregate function or the GROUP BY clause.

How can we achieve the desired output?

We can get the desired output in two ways.

Solution1:

One of the ways to get the desired output is by including all the aggregations in a subquery and then JOINING that subquery with the main query. The following example exactly does the same.

```

SELECT Name, Salary, Employees.Department,
       Departments.DepartmentTotals,
       Departments.TotalSalary,
       Departments.AvgSalary,
       Departments.MinSalary,
       Departments.MaxSalary
FROM Employees
INNER JOIN
( SELECT Department, COUNT(*) AS DepartmentTotals,
  SUM(Salary) AS TotalSalary,
  AVG(Salary) AS AvgSalary,
  MIN(Salary) AS MinSalary,
  MAX(Salary) AS MaxSalary
  FROM Employees
  GROUP BY Department) AS Departments
ON Departments.Department = Employees.Department

```

Once you execute the above query then you will get the desired output. But look at the number of T-SQL statements that we wrote.

Solution2:

The second way that is the most preferable way to get the desired output is by using the OVER clause combined with the **PARTITION BY** clause as shown in the below code.

```

SELECT Name,
       Salary,
       Department,

```

```
COUNT(Department) OVER(PARTITION BY Department) AS DepartmentTotals,  
SUM(Salary) OVER(PARTITION BY Department) AS TotalSalary,  
AVG(Salary) OVER(PARTITION BY Department) AS AvgSalary,  
MIN(Salary) OVER(PARTITION BY Department) AS MinSalary,  
MAX(Salary) OVER(PARTITION BY Department) AS MaxSalary  
FROM Employees
```

Once you execute the above code, you will get the output as expected.

In the next article, I am going to discuss the [ROW_NUMBER function in SQL Server](#) with examples. Here, in this article, I try to explain the need and use of the **OVER Clause in SQL Server**. I hope you enjoy this **OVER Clause in SQL Server with Examples** article.